

UNIVERSIDAD DE EL SALVADOR
FACULTAD MULTIDISCIPLINARIA DE OCCIDENTE
DEPARTAMENTO DE INGENIERÍA Y ARQUITECTURA

Ingeniería En Desarrollo De Software/Educación En Línea
CICLO II/2025

Asignatura: Programación Orientada a Objetos

Unidad 1: Programación y Principios de Diseño Orientado a Objetos.

Laboratorio práctico #2

Indicaciones: Complete cada una de las partes del laboratorio y haga el entregable correspondiente tal como se indica.

Temas a evaluar:

- Todos los temas desde 1.3 hasta 2.7.3, incluyendo Tipos de Datos, Clases, Objetos, Métodos, Herencia, polimorfismo, encapsulamiento, estructura de paquetes.

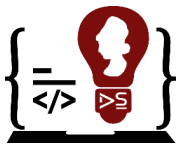
Objetivo del laboratorio: La tienda ha expandido su catálogo. Ahora vende productos de diferentes tipos, como electrónicos (con garantía) y alimentos (con fecha de vencimiento). Además, la política de impuestos ha cambiado: algunos productos tienen el IVA estándar, pero otros, como los de la canasta básica, están exentos de impuestos. El sistema actual es demasiado rígido para manejar esta diversidad. Es necesario un diseño que permita añadir nuevos tipos de productos e impuestos sin modificar el código principal.

Requisitos del programa: (Es importante que sigas estrictamente la estructura de paquetes, nombres de clases, atributos y métodos tal como se indica, de lo contrario tu nota podría verse afectada)

Rúbrica de evaluación:

- Nomenclatura de paquetes y clases 33.3%
- Requisitos Internos de clases 33.3%
- Compilación y funcionamiento 33.3%





A. Paquete `model` (Aplicando Herencia y Clases Abstractas)

1. Clase Abstracta `Producto.java`:

- La clase `Producto.java` del laboratorio anterior debe convertirse en una **clase abstracta**.
- Los atributos `sku`, `nombre`, `precio` y `cantidadStock` se mantienen.
- El constructor y los métodos `get/set` para estos atributos se mantienen.
- **Nuevo Método Abstracto:** Debes añadir el siguiente método abstracto, que forzará a las clases hijas a proporcionar detalles específicos sobre ellas:
Java

Java

```
public abstract String obtenerDetallesAdicionales();
```

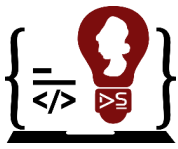
2. Clase `ProductoElectronico.java`:

- Debe **heredar** de la clase abstracta `Producto`.
- **Atributo Adicional:** `private int mesesGarantia;`
- **Constructor:** Debe recibir todos los parámetros (incluyendo los de la clase padre) y llamar a `super()` para inicializar los atributos heredados.
- **Implementación con `@Override`:** Debe implementar el método `obtenerDetallesAdicionales()`, devolviendo un `String` como:
"Garantía: 12 meses".

3. Clase `ProductoAlimenticio.java`:

- Debe **heredar** de la clase abstracta `Producto`.
- **Atributo Adicional:** `private String fechaVencimiento;`
- **Constructor:** Similar al de `ProductoElectronico`, debe recibir todos los parámetros y llamar a `super()`.





- Implementación con **@Override**: Debe implementar `obtenerDetallesAdicionales()`, devolviendo un String como: `"Vence: 2025-12-31"`.

B. Paquete **service** (Aplicando Interfaces y Polimorfismo)

1. Interfaz **ICalculadoraImpuesto.java**:

- Crea una interfaz que defina el contrato para cualquier cálculo de impuestos.
- **Método a definir**:
Java

Java

```
double calcularImpuesto(double valorTotal);
```

2. Clase **ImpuestoIVA.java** :

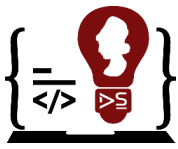
- Debe **implementar** la interfaz **ICalculadoraImpuesto**.
- **Constante**: `IVA = 0.13`.
- Implementación con **@Override**: El método `calcularImpuesto` debe recibir el valor total de los productos y devolver el monto correspondiente al 13% de IVA (`valorTotal * IVA`).

3. Clase **ImpuestoExento.java**:

- Debe **implementar** la interfaz **ICalculadoraImpuesto**.
- Implementación con **@Override**: El método `calcularImpuesto` debe simplemente devolver `0.0`, ya que estos productos no pagan impuestos.

4. Clase **ServicioInventario.java**:





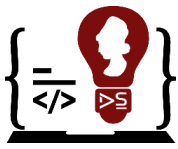
- La lista interna ahora debe ser de tipo `ArrayList<Producto>`, permitiendo almacenar cualquier objeto que herede de `Producto` (polimorfismo).
 - El método
 - `double calcularSubtotalInventario()`: Recorre los productos y devuelve la suma total de (`precio * cantidadStock`), es decir, el valor **antes de impuestos**.
-

C. Paquete `controller` (Demostración del Polimorfismo)

1. Clase `Main.java`:

- El método `main` demostrará la flexibilidad del nuevo sistema.
- **Lógica a implementar:**
 1. **Crear Instancias:** Instancia el `ServicioInventario` y las diferentes estrategias de impuesto (`ImpuestoIVA` e `ImpuestoExento`).
 2. **Crear Productos:** Crea e instancia al menos un `ProductoElectronico` y un `ProductoAlimenticio`.
 3. **Poblar Inventario:** Agrega estos productos al `ServicioInventario`.
 4. **Calcular Subtotal:** Llama al método `servicio.calcularSubtotalInventario()` y guarda el resultado.
 5. **Aplicar Estrategias (Polimorfismo en Acción):**
 - Calcula el impuesto usando la estrategia `ImpuestoIVA` sobre el subtotal. Calcula y muestra el total final (`subtotal + impuesto`).
 - Ahora, calcula el impuesto usando la estrategia `ImpuestoExento` sobre el **mismo subtotal**. Calcula y muestra el nuevo total final.





6. Imprime los detalles de un producto, incluyendo el resultado de llamar a `obtenerDetallesAdicionales()`.

Instrucciones de entrega (Git):

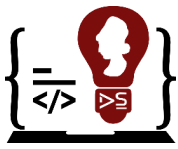
Paso 1: Aceptar la Tarea

1. Haz clic en el enlace de invitación <https://classroom.github.com/a/9jqcahzB>
2. Se abrirá una página de GitHub Classroom. Inicia sesión con tu cuenta de GitHub.
3. Verás el nombre de la tarea y un botón verde grande que dice **"Accept this assignment"** (**Aceptar esta tarea**). Haz clic en él.
4. **Espera un momento.** GitHub Classroom está creando tu repositorio personal para la tarea. La página se actualizará automáticamente cuando termine (puede tardar hasta un minuto).
5. Una vez listo, verás un enlace a tu repositorio. El enlace se verá así:
`github.com/Nombre-De-La-Organizacion/nombre-de-la-tarea-tu-usuario`. Haz clic en ese enlace.

Paso 2: Clonar tu Repositorio

Ahora estás en la página de **tu propio repositorio privado** para la tarea. Necesitas descargar una copia a tu computadora.

1. Haz clic en el botón verde **< > Code**.
2. Asegúrate de que la pestaña **HTTPS** esté seleccionada y copia la URL que aparece.
3. Abre tu terminal (Git Bash, Command Prompt, etc.), navega a la carpeta donde guardas tus proyectos y ejecuta el comando `git clone`, pegando la URL que copiaste:
Bash



```
git clone
```

```
https://github.com/Nombre-De-La-Organizacion/nombre-de-la-tarea-tu-usuario.git
```

Paso 3: Realizar el Trabajo

1. Navega a la nueva carpeta que se acaba de crear:

Bash

```
cd nombre-de-la-tarea-tu-usuario
```

2. Abre el proyecto en tu editor de código preferido (IntelliJ, etc.).
3. Escribe el código necesario para cumplir con los requisitos del laboratorio.

Paso 4: Guardar y Subir tus Cambios

Una vez que hayas terminado y probado tu código, debes guardarlo en Git y subirlo a GitHub.

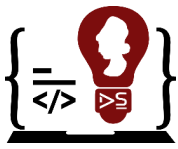
1. En tu terminal, añade tus cambios al área de preparación de Git:

Bash

```
git add .
```

2. Crea un "commit" para empaquetar tus cambios con un mensaje descriptivo:

Bash



```
git commit -m "Solución del Laboratorio 1"
```

3. Sube tus cambios a tu repositorio en GitHub:

Bash

```
git push
```

